

Tarea 02 Arquitectura Orientada a Microservicios

Lenguaje de Programación Avanzado 2

Estudiante	Guillermo Llano
Institución	Corporación Universitaria Remington
Actividad	Tarea 02 13 por ciento
Fecha de entrega	20 de septiembre de 2026

Este informe compara las arquitecturas monolítica y de microservicios y documenta una implementación práctica con dos servicios Spring Boot que se comunican mediante una API REST.

1 Cuadro comparativo

La elección arquitectónica depende del tamaño del sistema, la madurez del equipo y las necesidades reales de despliegue y escalabilidad.

Criterio	Arquitectura monolítica	Arquitectura de microservicios
Estructura	Una aplicación y una base de código concentran las funciones.	Servicios pequeños separados por capacidades de negocio.
Despliegue	Se publica nuevamente toda la aplicación.	Cada servicio puede desplegarse de manera independiente.
Escalabilidad	Se escala el sistema completo aunque solo una función tenga alta demanda.	Se escala únicamente el servicio que necesita más recursos.
Acoplamiento	Los módulos comparten proceso y suelen depender estrechamente entre sí.	Los servicios mantienen bajo acoplamiento mediante contratos API.
Tecnología	Normalmente se utiliza una pila tecnológica común.	Cada servicio puede elegir una tecnología adecuada, con control de estándares.
Datos	Es frecuente compartir una base de datos y transacciones locales.	Cada servicio es dueño de sus datos; la consistencia distribuida es más compleja.
Fallas	Un error grave puede afectar toda la aplicación.	Una falla puede aislarse, pero debe manejarse la caída de dependencias remotas.
Pruebas y depuración	El flujo se sigue dentro de un solo proceso.	Se requieren pruebas de contrato, integración y trazabilidad entre servicios.
Operación	Inicio y monitoreo más sencillos.	Exige automatización, observabilidad, seguridad y coordinación operativa.
Uso recomendado	Prototipos y aplicaciones pequeñas o de complejidad moderada.	Sistemas grandes, dominios diferenciados y equipos con capacidad DevOps.

2 Ensayo breve

Por qué las grandes empresas tecnológicas adoptan microservicios

Las grandes empresas tecnológicas operan aplicaciones que cambian de manera constante y atienden grandes volúmenes de usuarios. En ese contexto, una arquitectura monolítica puede convertirse en un límite organizacional y técnico: una modificación pequeña obliga a compilar, probar y desplegar el sistema completo, mientras todos los equipos trabajan sobre una misma unidad. Los microservicios responden a este problema al dividir la aplicación en capacidades de negocio con límites y contratos claros. Cada servicio puede evolucionar sin requerir que toda la plataforma sea publicada nuevamente.

La independencia de despliegue es una razón central para su adopción. Cuando un equipo es responsable de un servicio, puede desarrollar, probar y entregar cambios con un ciclo propio. Esto reduce la coordinación necesaria para cada versión y favorece entregas frecuentes. La separación también permite limitar el impacto de ciertos errores: si un servicio no esencial falla y el sistema fue diseñado con tolerancia adecuada, otras funciones pueden continuar disponibles. Sin embargo, esta resiliencia no aparece automáticamente; requiere tiempos de espera, manejo de errores, monitoreo y estrategias de recuperación.

Otro motivo es la escalabilidad selectiva. En una aplicación monolítica se aumentan recursos para todo el sistema incluso cuando la demanda se concentra en una sola función. Con microservicios, una empresa puede escalar únicamente búsquedas, pagos o reproducción de contenido, según el comportamiento real de sus usuarios. Esta asignación mejora el aprovechamiento de infraestructura y permite adaptar cada componente a cargas diferentes. La autonomía tecnológica también resulta útil: dentro de estándares de seguridad y operación, un servicio puede usar herramientas adecuadas a su necesidad sin forzar una migración total de la plataforma.

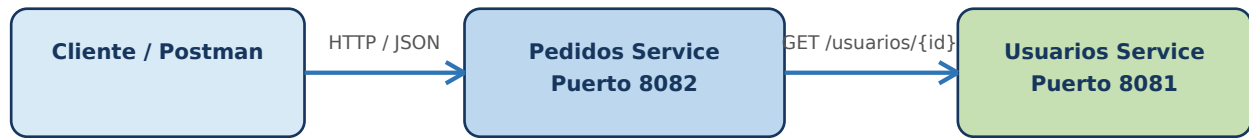
La arquitectura también acompaña la escala de los equipos. Un sistema dividido por dominios facilita que grupos pequeños asuman responsabilidad completa sobre componentes específicos. Esto reduce el conocimiento que cada desarrollador necesita mantener y permite que varios equipos trabajen en paralelo. Aun así, distribuir el software no elimina su complejidad: la traslada hacia la red, la observabilidad, la consistencia de datos, la automatización de despliegues y las pruebas de integración. Por ello, una organización necesita prácticas DevOps maduras y contratos API estables.

En conclusión, las grandes empresas adoptan microservicios porque necesitan autonomía, escalamiento selectivo y velocidad de cambio. La decisión no debe tomarse por moda. Para una aplicación pequeña, un monolito modular puede ser más simple y económico. Los microservicios se justifican cuando el dominio, el tráfico y la estructura de los equipos producen beneficios superiores al costo operativo de mantener un sistema distribuido.

Fuentes de apoyo: Fowler y Lewis (2014), AWS (2026) y Microsoft Learn (s. f.).

3 Diseño de la solución práctica

La solución utiliza dos aplicaciones Spring Boot independientes. Los datos se mantienen en memoria para enfocar la demostración en la separación de responsabilidades y la comunicación HTTP.



Pedidos valida el usuario mediante una llamada REST antes de guardar el pedido

Responsabilidades

Servicio	Puerto	Responsabilidad	Endpoints principales
Usuarios	8081	Listar, consultar y crear usuarios.	GET /usuarios; GET /usuarios/{id}; POST /usuarios
Pedidos	8082	Crear pedidos y asociarlos con un usuario consultado por REST.	GET /pedidos; POST /pedidos

Flujo de creación de un pedido

1. El cliente envía el producto, el valor y el identificador del usuario a Pedidos. 2. Pedidos consulta a Usuarios mediante GET /usuarios/{id}. 3. Si el usuario existe, Pedidos crea el registro y devuelve una respuesta HTTP 201 con los datos del usuario. 4. Si el usuario no existe o el servicio no está disponible, la operación devuelve un error controlado.

4 Evidencia del código

La implementación incluye validación de entradas, códigos HTTP apropiados y pruebas automatizadas de los controladores.

Comunicación REST desde Pedidos

```
public Usuario buscar(Long id) {
    try {
        return restClient.get()
            .uri(usuarioUrl + "/usuarios/" + id)
            .retrieve().body(Usuario.class);
    } catch (HttpClientErrorException.NotFound ex) {
        throw new ResponseStatusException(
            HttpStatus.BAD_REQUEST, "El usuario indicado no existe");
    }
}
```

Creación del pedido

```
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public Pedido crear(@Valid @RequestBody NuevoPedido entrada) {
    Usuario usuario = usuarioClient.buscar(entrada.usuarioId());
    Pedido pedido = new Pedido(secuencia.incrementAndGet(),
        entrada.producto(), entrada.valor(), usuario);
    pedidos.add(pedido);
    return pedido;
}
```

Solicitud de demostración

POST http://localhost:8082/pedidos
Content-Type: application/json

```
{
  "producto": "Teclado mecánico",
  "valor": 180000,
  "usuarioId": 1
}
```

Resultado esperado

```
HTTP 201 Created
{
  "id": 1,
  "producto": "Teclado mecánico",
  "valor": 180000,
  "usuario": {"id": 1, "nombre": "Ana Torres",
    "correo": "ana@ejemplo.com"}
}
```

5 Ejecución y repositorio

Requisitos: Java 17 y Maven 3.9 o superior. Se abren dos terminales desde la carpeta del proyecto.

```
Terminal 1  
cd usuarios-service  
mvn spring-boot:run
```

```
Terminal 2  
cd pedidos-service  
mvn spring-boot:run
```

```
Pruebas automatizadas desde la raíz  
mvn test
```

URL del repositorio GitHub: PENDIENTE REEMPLAZAR ANTES DE ENTREGAR

URL del video de sustentación: PENDIENTE REEMPLAZAR ANTES DE ENTREGAR

Archivos incluidos

El proyecto contiene ambos servicios, pruebas, una colección de Postman, README con instrucciones, texto del foro y guion del video. La colección permite ejecutar la demostración sin escribir manualmente cada solicitud.

Conclusiones

La práctica demuestra que los servicios pueden ejecutarse y evolucionar como procesos separados, comunicándose mediante un contrato REST. También evidencia una dificultad propia de los sistemas distribuidos: Pedidos depende de la disponibilidad y la respuesta de Usuarios. La arquitectura ofrece autonomía y escalabilidad, pero exige diseñar explícitamente el manejo de fallas y las pruebas entre servicios.

6 Participación en el foro

Respuesta a la pregunta sobre beneficios y desafíos

Considero que el principal beneficio de los microservicios es que permiten dividir una aplicación grande en servicios pequeños, cada uno responsable de una capacidad de negocio concreta. Esta separación facilita que diferentes equipos desarrollen, prueben, desplieguen y escalen sus componentes de forma independiente. Por ejemplo, si el servicio de pedidos recibe más tráfico que el de usuarios, se pueden asignar más recursos únicamente a pedidos. Además, una falla localizada no necesariamente detiene toda la aplicación y cada servicio puede evolucionar sin obligar a desplegar nuevamente el sistema completo.

Sin embargo, esta arquitectura también introduce desafíos importantes. La comunicación por red puede fallar o presentar demoras; por eso se necesitan validaciones, manejo de errores y mecanismos de observabilidad. También aumentan la complejidad del despliegue, la seguridad, las pruebas de integración y la consistencia de los datos distribuidos. En un monolito es más sencillo seguir una operación porque todo ocurre dentro del mismo proceso, mientras que en microservicios una solicitud puede recorrer varios sistemas. Por esta razón, no considero que los microservicios sean siempre la mejor opción. Son especialmente útiles en aplicaciones grandes, con necesidades diferentes de escalabilidad y equipos capaces de administrar infraestructura distribuida; para proyectos pequeños, un monolito bien organizado puede ser más económico y fácil de mantener.

Extensión aproximada: 190 palabras.

7 Referencias

Amazon Web Services. (2026). What's the difference between monolithic and microservices architecture?

<https://aws.amazon.com/compare/the-difference-between-monolithic-and-microservices-architecture/>

Fowler, M., y Lewis, J. (2014). Microservices. MartinFowler.com.

<https://martinfowler.com/articles/microservices.html>

Microsoft Learn. (s. f.). Challenges and solutions for distributed data management. <https://learn.microsoft.com/en-us/dotnet/architecture/microservices/architect-microservice-container-applications/distributed-data-management>

Spring. (s. f.). Building a RESTful Web Service. <https://spring.io/guides/gs/rest-service>